

Implementación de técnicas de Deep Learning Chatbot Conversacional

Diego Isaac Fuentes Juvera
A01705506

1. Introducción

Este documento tiene como objetivo describir el proceso de desarrollo y la arquitectura de mi proyecto de Deep Learning: un chatbot en inglés capaz de tener una conversación coherente en ciertos escenarios. El proyecto explora el uso de redes neuronales recurrentes (LSTM) para el procesamiento de lenguaje natural, implementando dos enfoques distintos de tokenización que resultaron en arquitecturas con capacidades y limitaciones muy diferentes.

2. Descripción del Dataset

Para el entrenamiento del modelo combiné 5 datasets de texto en inglés disponibles en Kaggle, buscando maximizar la diversidad y calidad de las conversaciones:

2.1. Datasets Utilizados

1. **Dataset de Conversaciones (3,724 ejemplos)**: Contiene ejemplos de conversación con formato pregunta-respuesta, incluyendo conversación informal, conversación formal, entrevistas, servicio al cliente, y conversaciones de redes sociales. Cada pregunta y respuesta da continuidad a la secuencia anterior, lo que ayuda al modelo a mantener coherencia contextual.
2. **Dataset para Chatbots**: Específicamente diseñado para entrenar chatbots con ejemplos de diferentes respuestas para varias preguntas, proporcionando variedad en los patrones de respuesta.
3. **Dataset de Conversaciones Humanas**: Contiene ejemplos reales de conversaciones entre dos personas, incluyendo tanto ejemplos de pregunta-respuesta como plática informal, lo que aporta naturalidad al modelo.
4. **Diálogos de South Park (150,000 caracteres)**: Dataset conformado por diálogos transcritos de episodios de South Park. Incluye el episodio, temporada, y el personaje que dice el diálogo. Para el entrenamiento solo utilicé los diálogos, eliminando los metadatos.

5. **Oraciones de Wikipedia (150,000 caracteres)**: Contiene oraciones aisladas extraídas directamente del contenido de artículos de Wikipedia, excluyendo datos de fichas técnicas o información que no sirva para entrenar modelos de lenguaje.

2.2. Construcción del Dataset Final

Junté todos los datasets en una sola secuencia de texto. Tomé los primeros 3 datasets completos ya que no eran muy grandes (aproximadamente 285,415 caracteres en total). De los datasets de Wikipedia y South Park tomé los primeros 150,000 caracteres de cada uno. El dataset final tiene una longitud de 585,415 caracteres, proporcionando suficiente diversidad para el entrenamiento sin ser excesivamente grande para los recursos computacionales disponibles.

3. Encoding y Tokenización

Uno de los aspectos más críticos del proyecto fue la elección del método de encoding, ya que determina cómo el modelo interpreta y genera texto. Implementé dos enfoques radicalmente diferentes que resultaron en arquitecturas con capacidades muy distintas.

3.1. Encoding a Nivel de Carácter

Para mis primeras versiones del modelo utilicé un encoding a nivel de carácter, contando únicamente con 114 tokens únicos (uno para cada carácter presente en el dataset). Este acercamiento es muy conveniente para ahorrar memoria y tiempo en el entrenamiento por varias razones:

- El modelo solo recibe de entrada 114 valores posibles
- El one-hot encoding resultante es un vector de apenas 114 dimensiones
- La capa de salida también es un vector de solo 114 valores
- Permite usar secuencias más largas (timesteps) sin exceder la memoria

Con este encoding, mi arquitectura podía ser más robusta:

- **Embedding**: 512 dimensiones
- **Timesteps**: 2048 caracteres de contexto
- **Epochs**: 40 iteraciones completas
- **Tiempo por epoch**: Aproximadamente 14 segundos

Sin embargo, este enfoque tiene limitaciones fundamentales. Al operar a nivel de carácter, el modelo debe aprender no solo las palabras, sino también cómo los caracteres se combinan para formar palabras con significado. Esto requiere que el modelo dedique capacidad de aprendizaje a patrones ortográficos básicos en lugar de enfocarse en patrones semánticos y conversacionales de nivel superior.

3.2. Encoding con Tiktoken (Nivel de Subpalabra)

Como segunda versión del encoding utilicé la librería Tiktoken, que implementa los tokenizadores utilizados por los modelos de código abierto de OpenAI. Elegí el tokenizador de GPT-2 (r50k_base) debido a que representa un punto medio entre robustez y complejidad:

- **Vocabulario:** 50,257 tokens diferentes
- **Nivel de tokenización:** Lexemas (raíz de la palabra) y morfemas (significado gramatical)
- **Ventaja:** El modelo tiene un entendimiento mucho mayor y más generalizado del texto

Este tokenizador funciona mediante Byte Pair Encoding (BPE), una técnica que identifica las secuencias de caracteres más frecuentes y las trata como unidades indivisibles. Por ejemplo, palabras comunes como "the", "is", o "hello" se representan con un solo token, mientras que palabras menos comunes se dividen en subpalabras conocidas.

3.3. Impacto en la Arquitectura

El cambio a Tiktoken tuvo un gran impacto en la arquitectura y derivó en un incremento exponencial en uso de recursos:

- La capa de embedding crece de 114 a 50,257 dimensiones de entrada
- La capa de salida debe generar probabilidades para 50,257 tokens posibles
- El número de parámetros en la capa densa final se multiplica por más de 400

Esto requirió reducir drásticamente otros aspectos del modelo para mantenerse dentro de las limitaciones de Kaggle (16GB de GPU RAM):

- **Embedding:** Reducido de 512 a 256 dimensiones
- **Timesteps:** Reducido de 2048 a 100 tokens de contexto
- **Batch size:** Reducido a 16 ejemplos por batch
- **Tiempo por epoch:** Incrementó de 14 segundos a aproximadamente 12 minutos (un aumento de más de 50x)

4. Arquitectura del Modelo

4.1. Componentes Fundamentales

El modelo implementa una arquitectura secuencial basada en redes neuronales recurrentes, específicamente diseñada para el procesamiento y generación de lenguaje natural.

4.1.1. Capa de Embedding

La capa de embedding es fundamental para transformar los tokens discretos en representaciones vectoriales continuas que el modelo puede procesar:

```
self.embedding = tf.keras.layers.Embedding(vocab_size, embedding_dim)
```

Esta capa aprende automáticamente durante el entrenamiento a representar cada token como un vector denso de `embedding_dim` dimensiones. Tokens semánticamente similares tienden a tener representaciones vectoriales cercanas en este espacio de alta dimensión, lo que permite al modelo capturar relaciones de significado.

4.1.2. Capa Recurrente LSTM

El corazón del modelo es una capa LSTM (Long Short-Term Memory), un tipo especializado de red neuronal recurrente diseñada para manejar dependencias a largo plazo:

```
self.lstm = tf.keras.layers.LSTM(rnn_units,
                                return_sequences=True,
                                return_state=True)
```

Configuración clave:

- **rnn_units**: 700 unidades ocultas, proporcionando capacidad suficiente para modelar patrones conversacionales complejos
- **return_sequences=True**: Devuelve una salida para cada timestep, no solo para el último, permitiendo predecir el siguiente token en cada posición
- **return_state=True**: Devuelve los estados ocultos (hidden state y cell state), esenciales para mantener el contexto en generación secuencial

El LSTM procesa secuencias mediante tres "puertas" especializadas:

- **Forget gate**: Decide qué información del cell state anterior debe olvidarse
- **Input gate**: Determina qué nueva información debe almacenarse
- **Output gate**: Controla qué información del cell state debe usarse para generar la salida

Esta arquitectura permite al modelo mantener información relevante durante muchos timesteps mientras descarta información irrelevante, crucial para mantener coherencia en conversaciones largas.

4.1.3. Capa Densa de Salida

La capa final es una red densa completamente conectada que proyecta la salida del LSTM al espacio de vocabulario:

```
self.dense = tf.keras.layers.Dense(vocab_size)
```

Esta capa produce logits (valores sin normalizar) para cada token posible en el vocabulario. Durante el entrenamiento, estos logits se comparan con los tokens reales mediante la función de pérdida. Durante la inferencia, se aplica softmax para convertir los logits en probabilidades y se muestrea el siguiente token.

4.2. Flujo de Datos en el Modelo

El método `call` define cómo los datos fluyen a través del modelo:

```
def call(self, inputs, states=None, return_state=False, training=False):
    x = inputs
    x = self.embedding(x, training=training)

    if states is None:
        lstm_output, hidden_state, cell_state = self.lstm(x, training=training)
    else:
        lstm_output, hidden_state, cell_state = self.lstm(x,
                                                         initial_state=states, training=training)

    states = [hidden_state, cell_state]
    x = lstm_output
    x = self.dense(x, training=training)

    if return_state:
        return x, states
    else:
        return x
```

Características importantes:

- **Gestión de estados:** Permite inicialización de estados para generación secuencial coherente
- **Modo training:** Controla comportamientos específicos de entrenamiento vs. inferencia
- **Return state:** Facilita la generación token por token manteniendo el contexto

4.3. Configuración Final del Modelo

Con el encoding de Tiktoken, la configuración final del modelo es:

- **Vocab size:** 50,257 tokens
- **Embedding dim:** 256 dimensiones
- **RNN units:** 700 unidades LSTM
- **Sequence length:** 100 tokens
- **Batch size:** 16
- **Total parámetros:** 50,775,549 (193.69 MB)
 - Embedding: 12,865,792 parámetros
 - LSTM: 2,679,600 parámetros
 - Dense: 35,230,157 parámetros

4.4. Experimentación con GRU

Durante el desarrollo también experimenté con GRU (Gated Recurrent Unit) como alternativa al LSTM. Las GRU son una variante simplificada de LSTM con solo dos puertas (reset y update) en lugar de tres, lo que resulta en:

Ventajas observadas:

- Menos parámetros (aproximadamente 25 % menos que LSTM)
- Entrenamiento más rápido (reducción de 15 % en tiempo por epoch)
- Menor uso de memoria

Desventajas observadas:

- Menor capacidad para mantener dependencias muy largas
- Convergencia ligeramente más lenta en términos de epochs
- Respuestas menos coherentes en conversaciones largas

Para este proyecto específico, el LSTM demostró mejor rendimiento general a pesar de su mayor costo computacional, especialmente en mantener coherencia contextual durante las conversaciones.

5. Entrenamiento

5.1. Preparación de Datos

Los datos se prepararon mediante ventanas deslizantes de 100 tokens, donde cada secuencia de entrada X corresponde a los tokens en posiciones [i:i+100] y la secuencia objetivo Y corresponde a [i+1:i+101], implementando así la tarea de predecir el siguiente token:

```
def text_to_sequences(text):
    ids = enc.encode(text)
    sequences = []
    for i in range(len(ids) - SEQUENCE_LENGTH):
        seq = ids[i:i+SEQUENCE_LENGTH+1]
        sequences.append(seq)
    return tf.data.Dataset.from_tensor_slices(sequences)

dataset = text_to_sequences(text)
dataset = dataset.map(lambda seq: (seq[:-1], seq[1:]))
dataset = dataset.shuffle(10000).batch(BATCH_SIZE, drop_remainder=True)
```

El shuffle con buffer de 10,000 asegura que el modelo no aprenda patrones falsos basados en el orden de los ejemplos de entrenamiento.

5.2. Función de Pérdida y Optimización

Se utilizó Sparse Categorical Crossentropy, apropiada para clasificación multiclase donde las etiquetas son enteros (IDs de tokens):

```
loss = tf.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(optimizer='adam', loss=loss)
```

El parámetro `from_logits=True` indica que el modelo produce valores sin normalizar, permitiendo mayor estabilidad numérica al combinar la operación softmax con el cálculo de la pérdida.

5.3. Callbacks y Gestión del Entrenamiento

Implementé un sistema de checkpoints para guardar el progreso del modelo:

```
checkpoint_callback = tf.keras.callbacks.ModelCheckpoint(
    filepath=checkpoint_prefix,
    save_weights_only=True)
```

Esto permite:

- Recuperar el entrenamiento si se interrumpe
- Comparar diferentes versiones del modelo
- Seleccionar el mejor modelo basándose en métricas de validación

5.4. Proceso de Entrenamiento

El modelo se entrenó por 20 epochs, pero solo completó 15 antes de detenerse debido a limitaciones de tiempo en Kaggle. El comportamiento observado durante el entrenamiento fue:

- **Epoch 1-3:** Descenso rápido de loss (de 10.8 a 0.094)
- **Epoch 4-6:** Estabilización del loss alrededor de 0.094-0.095
- **Tiempo por epoch:** 775 segundos (aproximadamente 12 minutos)
- **Ejemplos procesados:** 9,173 batches por epoch

6. Generación de Texto

6.1. Clase OneStep para Inferencia

Para la generación de texto implementé una clase especializada que envuelve el modelo entrenado:

```

class OneStep(tf.keras.Model):
    def __init__(self, model, encode, decode, temperature=1.0):
        super().__init__()
        self.model = model
        self.encode = encode
        self.decode = decode
        self.temperature = temperature

    @tf.function
    def generate_one_step(self, input_ids, states=None):
        predicted_logits, states = self.model(input_ids,
                                              states=states,
                                              return_state=True)

        predicted_logits = predicted_logits[:, -1, :]
        predicted_logits = predicted_logits / self.temperature
        predicted_id = tf.random.categorical(predicted_logits, num_samples=1)
        predicted_id = tf.squeeze(predicted_id, axis=-1)
        return predicted_id, states

```

6.2. Muestreo con Temperatura

El parámetro de temperatura controla la aleatoriedad de las predicciones:

- **Temperatura igual a 1.0:** Muestreo directo de la distribución de probabilidad
- **Temperatura menor a 1.0:** Predicciones más conservadoras y deterministas
- **Temperatura mayor a 1.0:** Predicciones más creativas y arriesgadas

La división por temperatura antes del muestreo modifica la distribución de probabilidades, haciéndola más uniforme (temperatura alta) o más concentrada (temperatura baja).

6.3. Generación Secuencial

El proceso de generación mantiene el estado del LSTM entre pasos para preservar el contexto:

```

states = None
generated = input_ids.numpy().tolist()[0]

for _ in range(200):
    next_id, states = one_step_model.generate_one_step(input_ids, states)
    next_id = int(next_id.numpy()[0])
    generated.append(next_id)
    input_ids = tf.constant([[next_id]], dtype=tf.int32)

```

Este enfoque de generación token por token con mantenimiento de estado permite al modelo generar texto coherente durante secuencias arbitrariamente largas.

7. Resultados

7.1. Modelo con tokenización a nivel de caracter

Este modelo fue entrenado durante 40 epochs y mostró resultados limitados, ya que el modelo solo es capaz de imitar cosas que aprecen lenguaje humano pero tiende mucho a inventar palabras, y no es capaz de generar oraciones coherentes en un contexto conversacional. Puede generar secuencias de palabras que pueden tener sentido pero son muy cortas y parecen producto del azar.

A pesar de que el entrenamiento fue muy rápido y pude entrenarlo por muchas más epochs, esto no mejoró significativamente la calidad del texto generado, estancándose en una pérdida de al rededor de 0.1

Ejemplo de generación en un intento de conversación:

hello!

Human 1: Wow!

How are you?

Auch make you defeated the Evily Gads?

Absolutely!

I can shot the game, fatass!

What do you mean?

People see the quest time in control.

And what do you think about that?

Oh come bring a game to riding a bug immigrant and Sauper in bitore already are invent

Thank you for your time.

100 Park was an amisenamity 70 gie signic vagina system in pass area only on the tenD.

10 mile Pur is a dark asteroid from the inner regions of the asteroid better thing to

Observaciones:

- El modelo imita el lenguaje humano.
- Genera palabras no existentes.
- No tiene coherencia contextual.
- Genera texto sin seguir un patron conversacional.

7.2. Modelo con tokenización Tiktoken

Los experimentos con el modelo entrenado por 14 epochs mostraron capacidad de generar texto con estructura coherente:

Ejemplo de generación con prompt "Hello, how are you?":

"Hello, how are you? Really good, really good. it's a great price. i think i'll buy both of them. i bought you a pair of pants. thank you. i hope they fit. i hope you kept the receipt. you think they won't fit? i think i've put on some weight..."

Observaciones:

- El modelo mantiene coherencia local en la conversación
- Genera oraciones gramaticalmente correctas
- Los cambios de tema son relativamente naturales
- Aparecen patrones conversacionales realistas (preguntas y respuestas)

7.3. Pruebas Interactivas

Implementé un sistema de chat interactivo para evaluar el modelo en tiempo real. Los resultados mostraron comportamientos interesantes:

Conversación de ejemplo:

You: What is happening in South Park?

Bot: what do you mean?

You: The TV show

Bot: him?

You: yes

Bot: .

You: Where is Cartman?

Bot: yes, i did.

Análisis del comportamiento:

- El modelo intenta mantener el contexto conversacional
- Las respuestas son cortas, típicas de conversación informal
- Ocasionalmente pierde el hilo de la conversación (respuestas como "him?..º zes, i did.")
- Reconoce el formato pregunta-respuesta

7.4. Limitaciones Observadas

A pesar de los resultados prometedores, identifiqué varias limitaciones importantes:

1. **Pérdida de coherencia temática:** El modelo tiende a derivar hacia temas genéricos después de pocos intercambios
2. **Respuestas genéricas:** Frecuentemente produce respuestas seguras como "what do you mean?".^o zes.^{en} lugar de contenido específico
3. **Falta de memoria a largo plazo:** Aunque mantiene contexto inmediato, olvida información de turnos anteriores en la conversación
4. **Repetición:** En conversaciones largas, el modelo tiende a repetir patrones de respuesta similares
5. **Falta de tokens:** Debido a que la tokenización no está ajustada al dataset específico, algunas palabras no son bien representadas, como nombres propios.
6. **Quiebre de secuencias:** No se usan tokens específicos para delimitar el inicio y final de cada diálogo, por lo que opté por detener la generación al llegar a un punto o signo de interrogación/exclamación. Esto limita la capacidad de generar respuestas más largas y elaboradas.

8. Análisis de Overfitting y Underfitting

8.1. Diagnóstico del Estado del Modelo

Basándome en las métricas de entrenamiento y el comportamiento en inferencia, el modelo presenta características de **ligero underfitting**:

Evidencia de underfitting:

- El loss se estabilizó alrededor de 0.094-0.095, sugiriendo que el modelo podría beneficiarse de más entrenamiento
- Las respuestas tienden a ser genéricas, indicando que el modelo no ha capturado completamente los patrones conversacionales complejos
- La curva de aprendizaje muestra que el modelo aún estaba mejorando cuando se detuvo el entrenamiento
- No se observó divergencia significativa entre loss de entrenamiento y validación

8.2. Causas del Underfitting

1. **Entrenamiento insuficiente:** Solo 6-14 epochs cuando el modelo planificaba 20-40
2. **Arquitectura limitada por recursos:** La reducción forzada de embedding_dim (512→256) y timesteps (2048→100) limita la capacidad del modelo
3. **Vocabulario excesivamente grande:** Con 50,257 tokens, el modelo necesita muchos más datos y epochs para aprender distribuciones precisas para cada token
4. **Dataset moderado:** 585,415 caracteres (100,000 tokens) es relativamente pequeño para un vocabulario de 50K tokens

8.3. Por qué no es Overfitting

A pesar de tener 50.7M parámetros, el modelo NO presenta overfitting claro:

- No hay divergencia entre métricas de entrenamiento y validación
- El modelo generaliza razonablemente a prompts no vistos
- Las respuestas son demasiado genéricas (no memorizadas) para ser overfitting
- No se observan respuestas que reproduzcan exactamente secuencias del dataset

8.4. Estrategias de Mejora

Para alcanzar un mejor ajuste del modelo en el futuro intentaría:

A corto plazo:

1. Completar los 20-40 epochs planificados
2. Implementar early stopping basado en validación para encontrar el punto óptimo
3. Ajustar el learning rate con schedule (ej. cosine decay)

A mediano plazo:

1. Aumentar el dataset a 1-5M tokens
2. Implementar tokens especiales para delimitar diálogos.
3. Experimentar con vocabularios más pequeños y mejor ajustados al dataset (10K-20K tokens)

A largo plazo:

1. Migrar a arquitecturas Transformer (más eficientes para contextos largos)
2. Implementar attention mechanisms
3. Fine-tuning desde modelos preentrenados

9. Conclusiones

9.1. Reflexión Personal

Este proyecto me ha enseñado que el desarrollo de modelos de deep learning es un proceso iterativo de experimentación, donde las limitaciones técnicas impulsan decisiones creativas. La necesidad de balancear capacidad del modelo, recursos computacionales y tiempo de entrenamiento es fundamental en aplicaciones reales, y ahora tengo la capacidad de estimar el costo de este tipo de proyectos y tomar decisiones más acertadas, permitiéndome establecer compromisos que sé que soy capaz de cumplir con mis futuros clientes.

Disfrute mucho todo el proceso de iterar en el modelo gracias al acompañamiento que me dieron mis profesores, pues cuando sentía que estaba estancado me dieron recomendaciones muy buenas de cómo mejorar el modelo dentro de las limitaciones de recursos

computacionales que tengo, ampliando mi conjunto de herramientas de aquí al futuro y haciendo que profundizara mucho más en aspectos técnicos al despertar mi curiosidad de forma genuina.

El resultado final, aunque modesto en comparación con chatbots comerciales, demuestra comprensión sólida de los principios fundamentales del procesamiento de lenguaje natural y proporciona una base excelente para futuras mejoras. La capacidad del modelo de generar texto coherente con solo 15 epochs sugiere que con más recursos y tiempo, podría alcanzar niveles de rendimiento significativamente superiores, y en su estado actual sigue siendo una herramienta entretenida de usar que me permite comprender cosas más allá del Machine Learning.